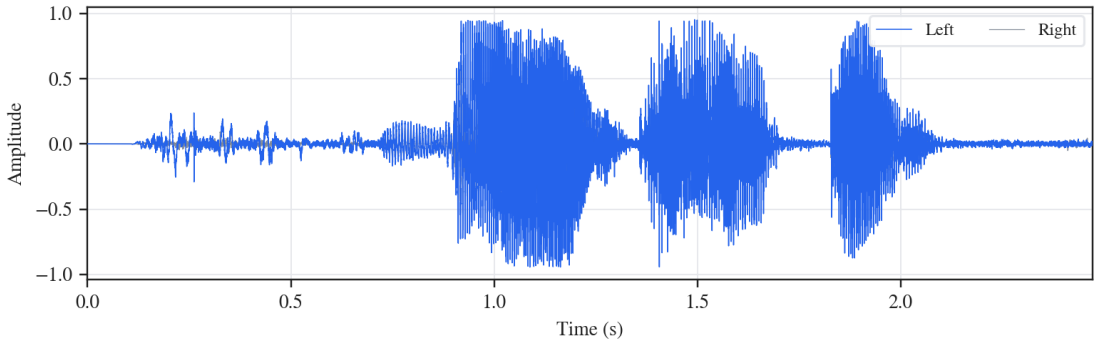
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
Birla Institute of Technology, Mesra

Digital Signal Processing of Speech Signals

Voice Recording, Silence Detection, Audio Preprocessing and Spectral Analysis

---



Waveform of the recording *Naman_BIT.wav* (left and right channels), computed from the original file.

Student	Naman Gupta
Roll number	BTECH/10887/24
Programme	B.Tech, Electronics and Communication Engineering, Section C
Course	EC24313 Artificial Intelligence and Machine Learning, Assignment 1 (Digital Signal Processing of voice recordings)
Submission date	17 October 2026

Abstract

This report analyses two voice recordings, one of the word “BIT” and one of the name “Naman Gupta”, with a reproducible Python pipeline. Both files are 16-bit stereo WAV recordings with a sampling frequency of 48,000 Hz read from the file headers; their durations are 2.472 s and 1.824 s. Time-domain waveforms were plotted, silent and low-energy regions were located with an adaptive frame-based energy detector that separates confident speech, silence and ambiguous low-level activity, the recordings were trimmed with padding and fades (to 1.690 s and 1.260 s, i.e. 31.6 % and 30.9 % removed), and short-time Fourier transform spectrograms were computed. The processed audio passed all applicable integrity checks and the code is covered by automated tests and by controlled synthetic experiments. Speech intelligibility of the processed audio, and the spoken content of the files, were not verified by listening; these and other limitations are stated in Section 10.

Contents

1 Introduction and Objectives . . . 3
1.1 Purpose . . . 3
1.2 Assignment objectives . . . 3
1.3 Tools . . . 3
2 Theoretical Background . . . 3
2.1 Sampling and discrete-time signals . . . 3
2.2 Amplitude measures . . . 3
2.3 Silence, unvoiced speech and background noise . . . 3
2.4 Frame-based analysis . . . 3
2.5 Short-Time Fourier Transform and the spectrogram . . . 4
3 Audio Acquisition and Sampling Frequency . . . 4
3.1 The recordings . . . 4
3.2 Obtaining the sampling frequency . . . 4
3.3 Channel handling and identity of the recordings . . . 5
4 Time-Domain Waveform Analysis . . . 5
4.1 Waveform plots . . . 5
4.2 Observations . . . 5
5 Silence and Unvoiced-Region Detection . . . 5
5.1 Algorithm . . . 5
5.2 Implementation . . . 6
5.3 Detected regions . . . 6
5.4 Silence versus unvoiced speech . . . 7
6 Audio Preprocessing and Trimming . . . 8
6.1 Procedure . . . 8
6.2 Implementation . . . 8
6.3 Results . . . 8
6.4 Trade-offs . . . 9
7 Spectrogram Analysis . . . 9
7.1 STFT configuration . . . 9
7.2 Spectrograms of the recordings . . . 9
7.3 Time-frequency resolution . . . 10
8 Implementation Overview . . . 10
9 Results, Validation and Discussion . . . 11
9.1 Real recordings (measured) . . . 11
9.2 Automated tests and joins . . . 11
9.3 Synthetic evaluation . . . 11
9.4 Recognition-based evidence and manual observations . . . 12
10 Limitations and Future Improvements . . . 12
11 Conclusion . . . 12
References . . . 12

1 Introduction and Objectives

1.1 Purpose

Speech is a non-stationary signal: its amplitude and its spectrum both change within tens of milliseconds, so it is analysed in the time domain (when energy is present) and in the time-frequency domain (which frequencies are present). A recorded file is only a sequence of amplitude samples; the sampling frequency fixes the time scale and the highest frequency that can be represented, silence at the start and end adds duration without information, and a spectrogram reveals voiced sounds, bursts and noise that a waveform cannot separate. This assignment applies these ideas to two recordings of the author's voice.

1.2 Assignment objectives

No.	Objective	Section
1	Record “BIT” and the student's name (*YOURNAME_BIT.wav*, *YOURNAME_NAME.wav*)	3
2	Plot the time-domain waveforms and determine the sampling frequency	3, 4
3	Identify and mark silent or unvoiced regions in the waveforms	5
4	Trim or minimise silent and flat regions without removing speech	6
5	Apply a time-frequency transformation and plot the spectrograms	7

1.3 Tools

Python 3 with NumPy (frame computations), SoundFile (WAV input and header metadata), SciPy (`scipy.signal.spectrogram` for the STFT), Matplotlib (plots), pytest (tests) and ReportLab (this document). Everything runs on a CPU without external services.

2 Theoretical Background

2.1 Sampling and discrete-time signals

A continuous-time speech signal $x_c(t)$ is measured every $T_s = 1/f_s$ seconds, giving the discrete-time signal $x[n]$ of Eq. (1). Sample n corresponds to the time of Eq. (2), where N is the number of samples per channel and T the duration. To avoid aliasing, f_s must exceed twice the highest frequency present (Nyquist criterion); frequencies up to $f_s/2$ can be represented. The sampling frequency is a property of the recording and must not be confused with the pitch (fundamental frequency) of the voice, which is roughly 85–255 Hz for adults.

$$x[n] = x_c(nT_s), \qquad T_s = \frac{1}{f_s} \tag{1}$$

$$t_n = \frac{n}{f_s}, \qquad T = \frac{N}{f_s} \tag{2}$$

2.2 Amplitude measures

Amplitudes are relative to digital full scale (range -1 to $+1$). The peak and the root-mean-square (RMS) amplitude are

$$A_{pk} = \max_n |x[n]|, \qquad A_{rms} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1} x^2[n]} \tag{3}$$

2.3 Silence, unvoiced speech and background noise

Exact digital silence (all-zero samples) is rare; a pause in speech still contains background noise, so “silence” here means a stretch whose short-time energy stays near the recording's noise floor. Unvoiced sounds (fricatives, plosive bursts) and breaths are also low in energy but are not silence, and quiet voiced sounds can lie only slightly above the noise. A fixed amplitude threshold therefore cannot be trusted; thresholds are set relative to each recording's own noise floor.

2.4 Frame-based analysis

The signal is divided into frames of L samples advancing by a hop H. The frame level used here pools the power of the C channels and is expressed in dB relative to full scale (ε avoids log 0):

$$L_m = 10\log_{10}\left(\frac{1}{CL}\sum_{c=1}^{C}\sum_{n=0}^{L-1} x_c^2[n+mH] + \varepsilon\right) \tag{4}$$

With a noise-floor level L_{noise} and a speech level L_{speech}, the detector uses a hold threshold and an onset threshold (Section 5):

$$T_{low} = L_{noise} + \Delta_{low}, \qquad T_{high} = \min(L_{noise} + \Delta_{high},\ L_{speech} - \Delta_{gap}) \tag{5}$$

2.5 Short-Time Fourier Transform and the spectrogram

A window $w[n]$ of length N_{w} slides along the signal with hop H, and each segment is Fourier transformed (m: frame index, k: frequency bin, centred at $f_k = k\,f_s/N_{\text{w}}$). The spectrogram shows the squared magnitude in dB relative to a reference power P_{ref}:

$$X(m,k) = \sum_{n=0}^{N_w-1} x[n+mH]\,w[n]\,e^{-j2\pi kn/N_w} \tag{6}$$

$$S_{dB}(m,k) = 10\log_{10}\left(\frac{|X(m,k)|^2}{P_{ref}}\right) \tag{7}$$

The window length trades frequency resolution against time resolution (Eq. 8): a long window separates harmonics but smears rapid changes. The Hann window reduces spectral leakage.

$$\Delta f = \frac{f_s}{N_w}, \qquad \Delta t_{window} = \frac{N_w}{f_s}, \qquad \Delta t_{hop} = \frac{H}{f_s} \tag{8}$$

3 Audio Acquisition and Sampling Frequency

3.1 The recordings

The repository contains two recordings of the author's voice: *Naman_BIT.wav*, declared to contain “BIT”, and *Naman_Name.wav*, declared to contain “Naman Gupta”. They follow the required *YOURNAME_BIT.wav* / *YOURNAME_NAME.wav* convention with YOURNAME = Naman. The files are used as originals: their SHA-256 hashes were checked before and after every processing run and did not change.

Table 1. Properties of the original recordings (WAV headers via SoundFile and decoded samples).

Property	Naman_BIT.wav	Naman_Name.wav
Declared utterance	BIT	Naman Gupta
Format / sample type	WAV / PCM_16	WAV / PCM_16
Sampling frequency	48,000 Hz	48,000 Hz
Channels	2 (stereo)	2 (stereo)
Samples per channel	118,656	87,552
Duration, N/f_s	2.472 s	1.824 s
Peak amplitude	0.9523 (-0.42 dBFS)	0.9477 (-0.47 dBFS)
RMS amplitude	0.1615 (-15.84 dBFS)	0.1666 (-15.56 dBFS)
Leading exact-zero samples	103.0 ms	103.0 ms

3.2 Obtaining the sampling frequency

The sampling frequency is read from the *fmt* chunk of the WAV header by SoundFile; it is not estimated from a plot. Both files have $\boldsymbol{f_s = 48{,}000}$ **Hz**, so the Nyquist frequency is 24,000 Hz and sample n occurs at $t = n/48000$ s (Eq. 2). Listing 1 shows the loading step.

```
def load_audio(path: Path) -> tuple[np.ndarray, int]:
    """Return float64 samples shaped (n_samples, n_channels) and the sampling
        frequency."""
    data, fs = sf.read(str(path), dtype="float64", always_2d=True)
    return data, int(fs)
# ...
info = sf.info(str(path))
"sampling_frequency_hz": fs,
```

Listing 1. Loading audio and header metadata (*assignment1.py*).

3.3 Channel handling and identity of the recordings

Both files are stereo. The waveform plots show each channel; the detector pools the power of both channels (Eq. 4), so a signal present in one channel only, or with opposite polarity in the two, is not lost as it would be by averaging. Spectrograms use the channel average, and trimming is applied to both channels together, so the processed files keep two channels at the original rate. The utterance in each file is a declaration recorded in *audio/manifest.json*; supporting recognition evidence is given in Section 9.4.

4 Time-Domain Waveform Analysis

4.1 Waveform plots

Figure 1 plots amplitude against time for both recordings. The time axis is built from the sample index and the sampling frequency exactly as in Eq. (2) (Listing 2).

```
i0 = int(t0 * fs)
i1 = data.shape[0] if t1 is None else int(t1 * fs)
t = np.arange(i0, i1) / fs
# ...
ax.plot(t, data[i0:i1, c], lw=0.6, color=colors[c % 2], alpha=0.85 if c == 0 else 0.6,
    label=label)
```

Listing 2. Time axis and waveform plotting (*assignment1.py*, `_plot_channels`).

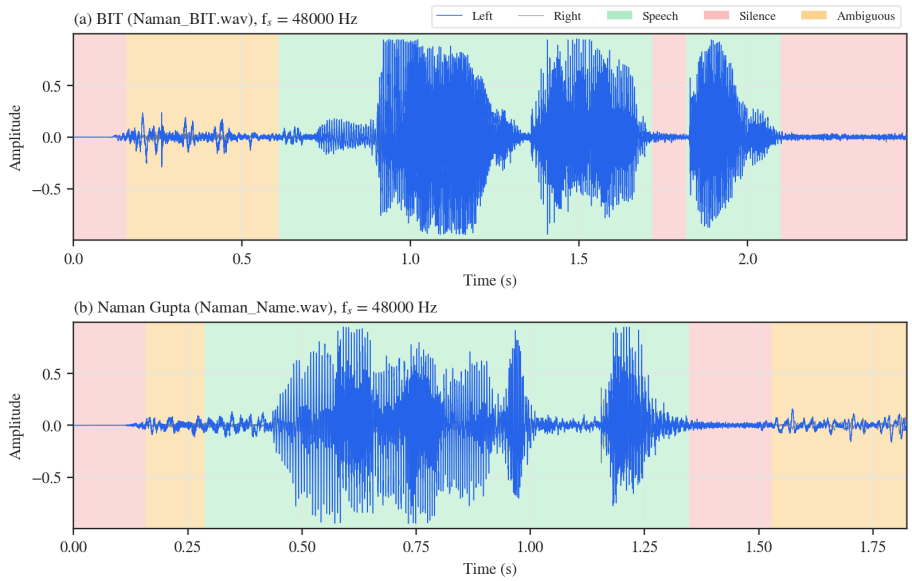


Figure 1. Time-domain waveforms of (a) *Naman_BIT.wav* and (b) *Naman_Name.wav*: amplitude (full scale = 1) against time (s), left channel in blue and right channel in slate, with detected speech (green), silence (red) and ambiguous low-level (amber) regions shaded.

4.2 Observations

Both recordings begin with a perfectly flat stretch of exact zeros (103.0 ms and 103.0 ms), followed by low-level noise at about -42.6 dBFS (BIT) and -42.4 dBFS (Naman Gupta), whereas the speech level is about -9.6 and -9.7 dBFS. The peaks of 0.952 and 0.948 show that neither file is clipped, and the left channel is generally stronger. In Fig. 1(a) the strong speech lies in two stretches (0.61–1.72 s and 1.82–2.10 s) separated by a dip to the noise floor, preceded by a low-amplitude lead-in of about 0.45 s (peak 0.291 of full scale) and followed by a flat noise floor. In Fig. 1(b) the strong speech runs from about 0.29 s to 1.35 s, followed by a quiet stretch and slow low-amplitude excursions in the last 0.3 s (peak 0.162). The waveform shows when energy is present and how large it is, but it cannot say whether a low-amplitude stretch is breath, noise or weak speech.

5 Silence and Unvoiced-Region Detection

5.1 Algorithm

Silent regions are identified by a frame-based energy detector, `detect_silence()`, which makes a three-way decision: *confident speech*, *silence* and *ambiguous low-level activity*. The steps are:

- **Frame level** (Eq. 4): 25 ms frames, 10 ms hop (1200 and 480 samples at 48 kHz), power pooled over both channels.
- **Noise floor and speech level:** the noise floor is the median of the quietest 10 % of frames, excluding digital-zero frames; the speech level is the 95th percentile.
- **Hysteresis thresholds** (Eq. 5): hold threshold at the noise floor + 8 dB, onset threshold at the noise floor + 20 dB and at least 10 dB below the speech level. If speech is less than 15 dB above the noise, no region is declared silent.
- **Runs:** frames above the hold threshold form runs; dips shorter than 100 ms are bridged and runs shorter than 30 ms are discarded.
- **Confident speech:** a run needs at least 30 ms above the onset threshold and extends at most 150 ms beyond it (soft onsets and decays); other activity is ambiguous.
- **Periodicity check:** the high-energy core must contain at least 30 ms of periodic frames (normalised autocorrelation peak ≥ 0.5, lags for 70–400 Hz); otherwise it is labelled ambiguous. This tests for voicing, not for speech.

5.2 Implementation

```
valid = db > cfg.noise_exclude_db
dv = db[valid] if valid.sum() >= 5 else db
k = max(3, int(round(cfg.noise_fraction * dv.size)))
quiet = np.sort(dv)[:k]
noise = float(np.median(quiet))
mad = float(1.4826 * np.median(np.abs(quiet - noise)))
level = float(np.percentile(dv, cfg.speech_percentile))
low = noise + cfg.low_margin_db
high = max(min(noise + cfg.high_margin_db, level - cfg.level_gap_db), low + 3.0)
# ...
active = db > low
n = active.size
min_sil = max(1, int(round(cfg.min_silence_ms / cfg.hop_ms)))
min_sp = max(1, int(round(cfg.min_speech_ms / cfg.hop_ms)))
min_core = max(1, int(round(cfg.min_core_ms / cfg.hop_ms)))
for s, e in _runs(~active):            # bridge short dips between active frames
    if e - s < min_sil and s > 0 and e < n:
        active[s:e] = True
# ...
cores = [(a, b) for a, b in _runs(db[s:e] > high) if b - a >= min_core]
if periodic is not None and cores:
    kept = [(a, b) for a, b in cores if int(periodic[s + a:s + b].sum()) >= min_per]
    n_nonperiodic += len(cores) - len(kept)
    cores = kept
```

Listing 3. Core of `detect_silence()`: noise-floor and threshold estimation, bridging of short dips, and periodicity-checked cores (*assignment1.py*).

5.3 Detected regions

Figure 2 marks the detected regions on the waveforms (speech in green, silence in red, ambiguous low-level activity in amber) and shows the frame level with the noise floor and both thresholds; Table 2 lists the intervals.

Table 2. Detected regions (from *outputs/analysis_results.json*).

Recording	Region type	Start (s)	End (s)	Duration (s)
BIT	silence (leading)	0.000	0.158	0.158
BIT	ambiguous low-level	0.158	0.608	0.450
BIT	confident speech	0.608	1.718	1.110
BIT	silence (internal)	1.718	1.817	0.100
BIT	confident speech	1.817	2.098	0.280
BIT	silence (trailing)	2.098	2.472	0.374
Naman Gupta	silence (leading)	0.000	0.158	0.158
Naman Gupta	ambiguous low-level	0.158	0.287	0.130
Naman Gupta	confident speech	0.287	1.347	1.060
Naman Gupta	silence (internal)	1.347	1.528	0.180
Naman Gupta	ambiguous low-level	1.528	1.824	0.296

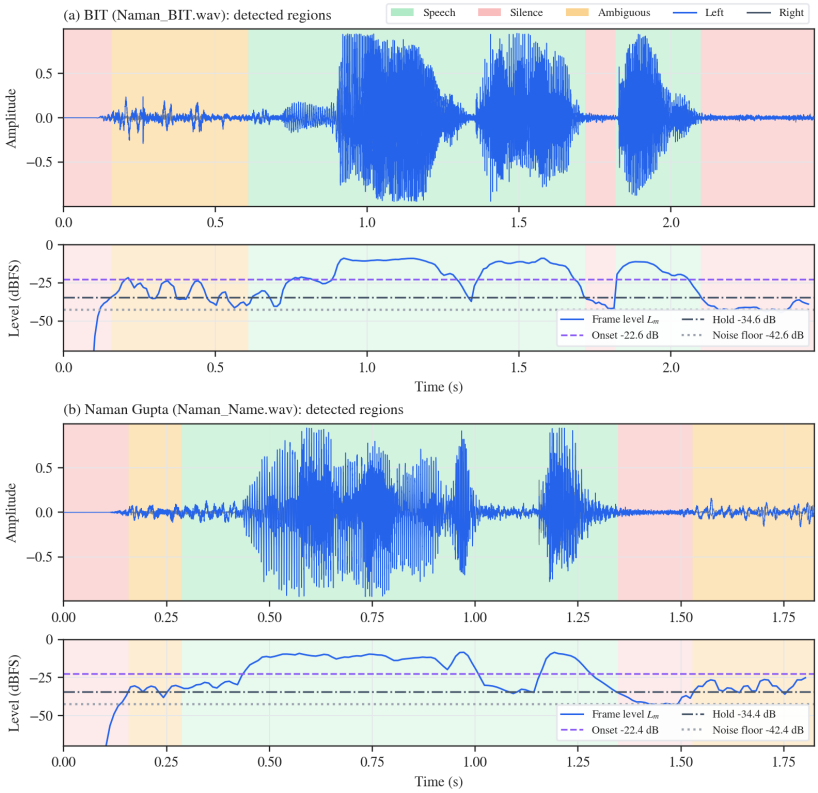


Figure 2. Detected regions for (a) *Naman_BIT.wav* and (b) *Naman_Name.wav*. Upper panels: waveform with speech (green), silence (red) and ambiguous (amber) regions. Lower panels: frame level L_m (blue) with the onset (purple dashed), hold (slate dash-dot) and noise-floor (gray dotted) levels.

5.4 Silence versus unvoiced speech

A region is called silence only relative to its recording's noise floor. A region above the noise floor that never reaches the onset threshold is reported as ambiguous, because breath, background noise, weak voicing and unvoiced consonants all look alike to an energy-based method. The detector therefore does not claim to classify phonemes. Three ambiguous regions were found: BIT 0.158–0.608 s, and Naman Gupta 0.158–0.287 s and 1.528–1.824 s. Their levels are 17–18 dB below the confident speech, they have borderline periodicity (43–62 % of frames) and most of their energy lies below 300 Hz, so automated features could not classify them. The author's listening check, recorded in *manual_listening.json*, reported all three as noise; the setup and date were not recorded and the pipeline cannot verify it.

6 Audio Preprocessing and Trimming

6.1 Procedure

- **Boundaries:** the output starts at the first and ends at the last confident-speech region, each extended by **100 ms** of padding (clamped to the file limits); the padding exceeds the 25 ms frame so soft onsets and decays survive.
- **Fades:** a 5 ms linear fade is applied at every cut that is not the original file edge, avoiding clicks.
- **Internal pauses:** a gap is shortened (to 200 ms) only if it is longer than 400 ms and both neighbours are confident speech. The internal silences here (0.099 s and 0.181 s) are shorter, so they were preserved and no joins were created.
- **Ambiguous regions:** controlled by `trim_ambiguous`. With **False** they are kept; with **True** (the current default) they are removed. The default follows the listening finding above.
- **Format:** the sampling frequency (48,000 Hz), the two channels, the 16-bit PCM format and the sample values are preserved; no gain is applied; the original files are never modified.

6.2 Implementation

```
pad = int(round(cfg.edge_pad_ms * fs / 1000))
start = max(0, int(round(active[0]["start_s"] * fs)) - pad)
end = min(n, int(round(active[-1]["end_s"] * fs)) + pad)
# ...
fade = int(round(cfg.fade_ms * fs / 1000))
pieces, joins, total = [], [], 0
for i, (s, e) in enumerate(keep):
    seg = data[s:e].copy()
    fade_in = i > 0 or s > 0
    fade_out = i < len(keep) - 1 or e < n
    if seg.shape[0] > 2 * fade > 0:
        if fade_in:
            seg[:fade] *= np.linspace(0, 1, fade)[:, None]
        if fade_out:
            seg[-fade:] *= np.linspace(1, 0, fade)[:, None]
    if i > 0:
        joins.append(total)
    total += seg.shape[0]
    pieces.append(seg)
out = np.concatenate(pieces, axis=0)
```

Listing 4. Boundary selection with padding, and fades at every cut (*assignment1.py*, `trim_silence`).

6.3 Results

Table 3. Duration reduction for the real recordings. The last two rows show, for comparison, the result of the original detector and of the conservative setting.

Quantity	BIT	Naman Gupta
Original duration (s)	2.472	1.824
Processed duration (s)	1.690	1.260
Removed duration (s)	0.782	0.564
Percentage removed	31.6 %	30.9 %
Original samples per channel	118,656	87,552
Processed samples per channel	81,120	60,480
Output covers original time (s)	0.5075 to 2.1975	0.1875 to 1.4475
Original single-threshold detector: processed (s) / removed	2.110 / 14.6 %	1.581 / 13.3 %
Ambiguous regions kept: processed (s) / removed	2.140 / 13.4 %	1.766 / 3.2 %

Figure 3 shows the original and trimmed waveforms on identical axes. **Integrity checks.** Each processed file was re-opened and verified: it opens, is non-empty, has only finite non-zero samples, keeps the sampling frequency and channel count, has the reported sample count, equals the in-memory result to within one 16-bit step, has no clipped samples and no peak above the original, and the original file's hash is unchanged. **All 11 applicable checks passed for both files.** Processed RMS is higher (0.194 and 0.200 versus 0.162 and 0.167) only because low-energy frames were removed from the average.

6.4 Trade-offs

Aggressive trimming shortens the file but risks cutting soft onsets, decays and weak consonants; conservative trimming keeps non-speech noise. The padding, the 150 ms onset extension and the ambiguity setting are safeguards: with ambiguous regions kept, the reductions are 13.4 % and 3.2 %; with them removed, 31.6 % and 30.9 %. Because the boundaries inherit the detector's uncertainty and the processed audio has not been listened to, intelligibility was not verified.

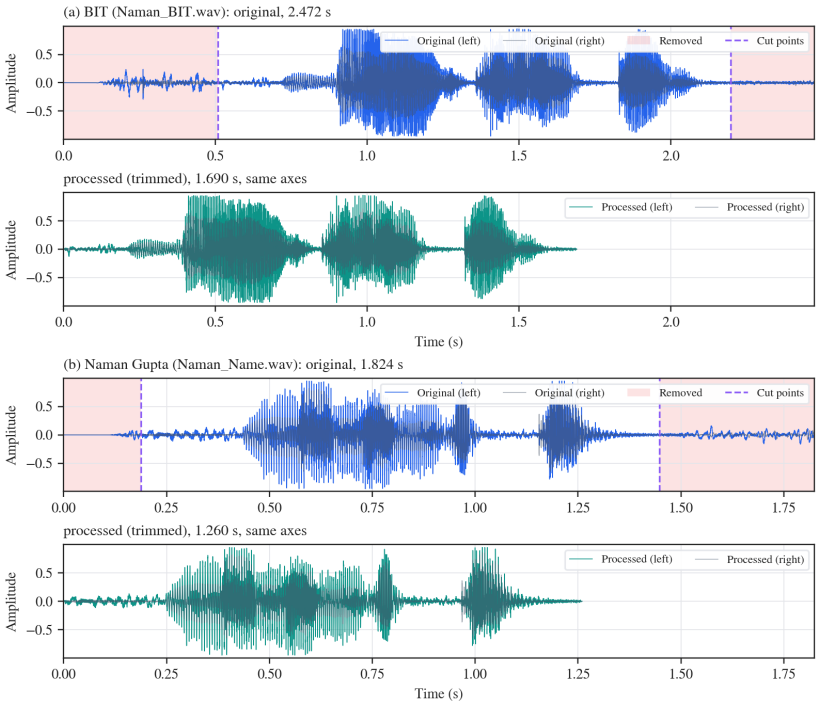


Figure 3. Original and trimmed recordings on identical axes: (a) *Naman_BIT.wav*, (b) *Naman_Name.wav*. The original is blue and the processed audio teal; purple dashed lines mark the cut points and red shading the removed material.

7 Spectrogram Analysis

7.1 STFT configuration

The transformation is the STFT of Eq. (6), computed with `scipy.signal.spectrogram` on the channel average: Hann window, $N_w = 1024$ samples, hop $H = 128$ samples (87.5 % overlap) and power-spectrum scaling. At 48,000 Hz (Eq. 8) the window is 21.33 ms long, the bin spacing is 46.875 Hz and the hop is 2.67 ms. The vertical axis is limited to 8000 Hz (below the 24,000 Hz Nyquist frequency), and the colour scale (magma, brighter meaning more power) is the power in dB relative to the peak power of the original recording (Eq. 7) over an 80 dB range.

```
nperseg = int(min(nfft or cfg.spec_nfft, x.size))
h = max(1, min(hop or cfg.spec_hop, nperseg // 2))
f, t, sxx = signal.spectrogram(x, fs=fs, window="hann", nperseg=nperseg,
    noverlap=nperseg - h,
    scaling="spectrum", mode="psd")
# ...
db = 10 * np.log10(sxx / ref + EPS)
```

Listing 5. STFT and decibel conversion (*assignment1.py*).

7.2 Spectrograms of the recordings

Time runs along the horizontal axis, frequency along the vertical axis and brightness shows power. Voiced sounds have a fundamental and harmonics that appear as horizontal stripes spaced at the pitch, vowels as broad bright bands over those stripes, and bursts or fricatives as broadband vertical or high-frequency patches. In Fig. 4(a) the voiced parts show harmonic stripes with strong low-frequency energy, vowel-like bands that change with time, and narrow broadband vertical lines near 0.9 s, 1.35 s and 1.83 s consistent with bursts; the first 0.1 s is uniformly dark (exact zeros), and the lead-in before the first vowel shows faint broadband energy with a click-like line near 0.26 s and no clear harmonic stripes. In Fig. 4(b) a broadband burst near 1.15 s is followed by a voiced segment with harmonics below about 2 kHz, and the final 0.3 s contains weak energy mostly below about 300 Hz without word-like structure. These are visual observations that agree with the amplitude analysis; they do not identify phonemes.

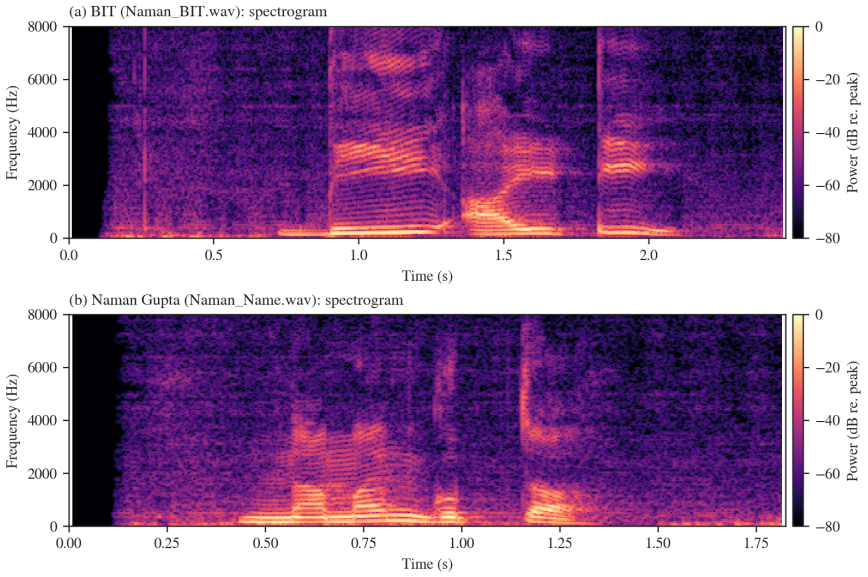


Figure 4. Spectrograms of (a) *Naman_BIT.wav* and (b) *Naman_Name.wav*: time (s), frequency (Hz) and power (dB relative to the recording's peak; brighter is stronger).

7.3 Time-frequency resolution

With 512 samples (10.7 ms) timing is sharp but neighbouring harmonics blur; with 2048 samples (42.7 ms) the harmonics are crisp but timing smears (Fig. 5). The 1024-point window (46.9 Hz bin spacing separates harmonics of a 100–200 Hz voice by two to four bins) was kept as a compromise, judged by inspection. Pitch varies within a window and the recordings are only 1–2 s long, so individual harmonics and formants should not be over-interpreted.

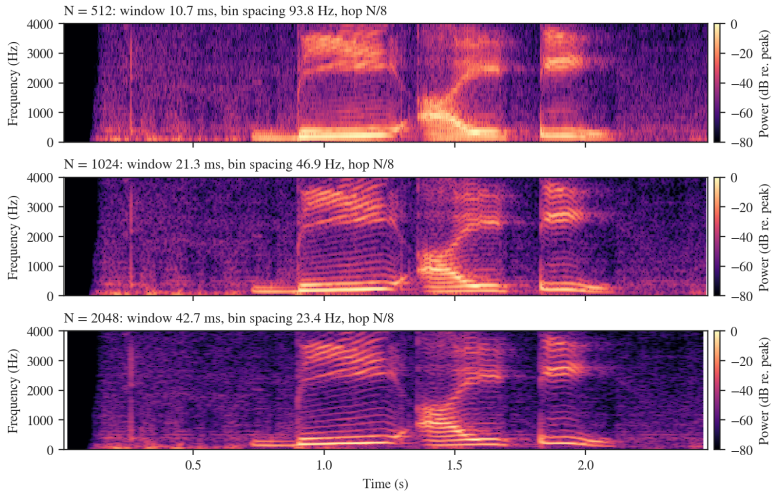


Figure 5. Effect of the window length N_w on *Naman_BIT.wav* (0–4 kHz, hop = N_w/8).

8 Implementation Overview

The project is a small modular Python package run by `python assignment1.py`. All parameters live in one `Config` dataclass, so no source edit is needed to change them.

Files. `assignment1.py` holds loading, metadata, the baseline and improved detectors, trimming, integrity checks, plotting, spectrograms and the command line; `evaluation.py` and `evaluation_events.py` hold the synthetic ground-truth evaluation, non-speech experiments and join validation; `regions.py` analyses ambiguous regions and prepares listening material; `verify_content.py` is an optional offline speech-recognition cross-check; `report.py` and `generate_assignment_pdf.py` write the Markdown report and this PDF; `tests/test_assignment1.py` contains the automated tests.

Pipeline. For each recording the program loads the audio and metadata (Listing 1), computes peak and RMS, detects silence (Listing 3), trims (Listing 4), draws the plots and spectrograms (Listings 2 and 5), writes the processed WAV at the original sampling frequency, re-reads and validates it, and finally writes *analysis_results.json* and the Markdown report. Reruns overwrite generated files only, and the original recordings are never written.

9 Results, Validation and Discussion

9.1 Real recordings (measured)

Both files were analysed end to end: the 48,000 Hz sampling frequency was read from the headers, silent and ambiguous regions were located, the recordings were shortened from 2.472 s to 1.690 s and from 1.824 s to 1.260 s (Tables 1 and 3), and spectrograms were produced. **No precision, recall or F1 is reported for these real recordings**, because no independently labelled speech/silence ground truth exists.

9.2 Automated tests and joins

All 41 automated tests passed (executed while generating this report). The tests cover loading and duration, mono and stereo handling, silence intervals on signals with known gaps, hysteresis behaviour, preservation of low-amplitude speech, trimming without modifying the input, empty/short/silent/invalid inputs, spectrogram limits, processed-file integrity, reruns and original-file preservation. No internal gap was shortened, so the real outputs contain **no joins** and join quality is not applicable to them; on a synthetic signal the fade reduced the sample step at a join from 0.83 to 0.00, which shows numerical continuity only, not perceptual smoothness.

9.3 Synthetic evaluation

Table 4. Synthetic evaluation of the original single-threshold detector (baseline) and the hysteresis detector (improved): frame-level F1 on a 10 ms grid and fraction of true speech samples preserved by trimming. These are generated signals, not the recordings.

Synthetic scenario (16 kHz, known truth)	Baseline F1	Improved F1	Baseline preserved	Improved preserved
clean	0.99	0.99	1.000	1.000
snr 20dB	1.00	1.00	1.000	1.000
snr 10dB	0.00	0.64	1.000	1.000
little silence (speech fills file)	0.48	1.00	0.981	1.000
stereo opposite polarity	0.00	0.99	1.000	1.000
noise burst 300ms (-34dBFS) in silence	0.86	0.86	1.000	1.000

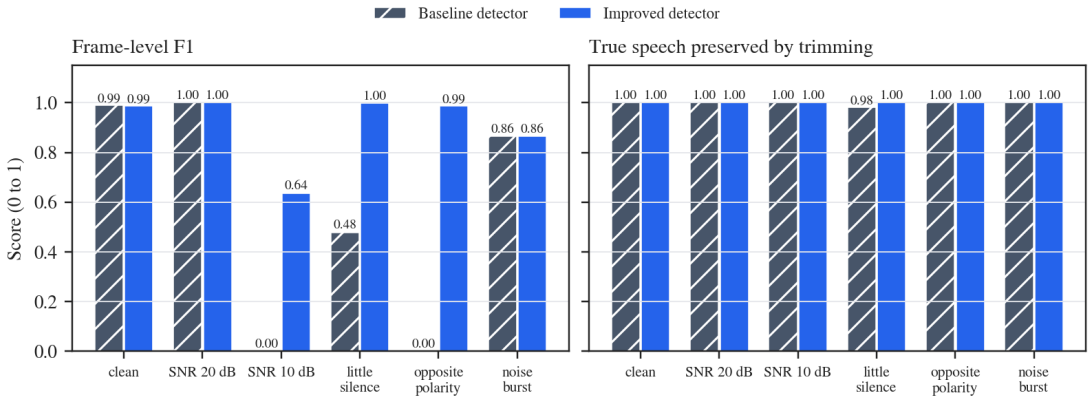


Figure 6. Synthetic evaluation, baseline (slate, hatched) versus improved (blue) detector for the scenarios of Table 4: frame-level F1 (left) and fraction of true speech samples preserved by trimming (right). Generated signals, not the recordings.

Table 5. Synthetic loud non-speech sounds: share of non-speech frames accepted as confident speech. The periodicity check helps for clicks, noise bursts and breath-like noise but not for a periodic thump.

Synthetic non-speech signal	Energy only	With periodicity check
loud click train	0.16	0.01
loud white noise burst	0.21	0.02
breath like noise (-26dBFS)	0.25	0.01
low frequency thump (80Hz)	0.16	0.16

The improved detector recovers speech in a file with little silence and in a file with opposite-polarity channels, where the baseline fails, and preserved all true speech in every scenario; at 10 dB SNR it cannot separate speech from noise and trims nothing. A loud click or noise burst inside silence is still treated as activity by both detectors, so energy-based detection cannot separate loud noise from speech.

9.4 Recognition-based evidence and manual observations

Table 6. Recognition evidence (local offline engines; supporting evidence only, not ground truth).

Recording	Declared	Whisper (offline)	Windows System.Speech (offline)	Status
Naman_BIT.wav	BIT	“BIT”	“The FBI B”	partially supported
Naman_Name.wav	Naman Gupta	“Naman Gupta”	“Non little”	partially supported

A local Whisper model transcribed both files as declared, whereas the Windows built-in recogniser, which is weak on isolated short words, returned unrelated text; the evidence is therefore partial and the spoken content is not confirmed by listening. The author's listening check of the ambiguous regions is a manual observation, not a measurement.

10 Limitations and Future Improvements

- Detection is energy-based with an optional voicing check; it is not a validated speech/non-speech classifier. Loud non-speech sounds (clicks, noise bursts, a periodic thump) can look like speech, quiet speech near the noise floor can be mislabelled, and the noise floor assumes the recording contains noise-only frames.
- Unvoiced sounds are not reliably separated from noise. An isolated unvoiced sound is labelled ambiguous by the periodicity check and would be lost if ambiguous regions are removed (13 % of its samples in a synthetic test); the removal setting is justified here only by the author's listening finding for these two files.
- The spoken content is only partly supported by recognition, and neither the processed audio nor its joins have been listened to. Synthetic results may not transfer to other recordings, and no accuracy figure exists for the two real recordings.

Possible improvements (not implemented): labelled real speech for quantitative evaluation, a structured multi-listener listening test, features validated on real data, and adaptive padding.

11 Conclusion

1. Recording: two 16-bit stereo recordings, *Naman_BIT.wav* and *Naman_Name.wav*, were used unmodified; their content is declared and partly supported by offline recognition.

2. Waveform and sampling frequency: amplitude–time plots were produced (Fig. 1); the sampling frequency is 48,000 Hz, read from the WAV header and not the voice pitch.

3. Silence identification: leading, trailing and internal silences and ambiguous low-level regions were detected adaptively and marked (Fig. 2, Table 2).

4. Preprocessing: silence was trimmed with padding and fades (31.6 % and 30.9 % removed), preserving the sampling frequency and channels, with all integrity checks passed (Fig. 3, Table 3).

5. Spectrograms: Hann-window STFT spectrograms (1024 samples, hop 128, 0–8 kHz, dB scale) were produced for both recordings, with an FFT-size comparison (Figs. 4 and 5).

References

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.

[2] L. R. Rabiner and R. W. Schafer, *Theory and Applications of Digital Speech Processing*, Pearson, 2011.

[3] F. J. Harris, “On the use of windows for harmonic analysis with the discrete Fourier transform,” *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, 1978.

[4] SciPy documentation, *scipy.signal.spectrogram*. https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.spectrogram.html

[5] NumPy documentation. https://numpy.org/doc/stable/ SoundFile documentation. https://python-soundfile.readthedocs.io/ Matplotlib documentation. https://matplotlib.org/stable/

[6] ReportLab documentation. https://docs.reportlab.com/

[7] A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” arXiv:2212.04356, 2022. faster-whisper: https://github.com/SYSTRAN/faster-whisper

Source code repository: https://github.com/Naman21036/voice-signal-analysis